Nombre: Grilmar Adrian Marroquin Vargas

Carne: 202506479

Asignatura: Estructura de Datos

Catedrático: Ing. Brandon Chitay

Fecha: 03/03/2026

Enlace de video: https://youtu.be/5myJTRzTLcg

Enlace de GitHub: https://github.com/adrianmarroquin560/ED_PARCIAL_-GRILMARADRIANMARROQUINVARGAS-

Introducción

Definición breve

Big-O es una notación matemática que describe cómo crece el tiempo o el espacio que usa un algoritmo cuando aumenta el tamaño de la entrada n. Se usa para expresar el orden de crecimiento (una cota superior) y comparar la eficiencia asintótica de algoritmos sin depender de detalles de implementación o del hardware

Factorial — Iterativo

Se calcula multiplicando 1×2×3…×n dentro de un bucle. Es directo: cada vez que aumentas $n$haces una multiplicación más, por eso el tiempo crece de forma proporcional a n.

Factorial — Recursivo

La función se llama a sí misma hasta llegar a 1 (por ejemplo, fact(n) = n * fact(n-1)). Hace lo mismo que el iterativo pero usando llamadas en pila; para los $n$que probaste los tiempos son parecidos al iterativo.

Fibonacci — Iterativo

Construye la serie desde abajo: guarda los dos últimos números y los suma en un bucle hasta llegar a n. Es rápido y escala de forma lineal con n, por eso tus mediciones se mantienen bajas.

Fibonacci — Recursivo

Cada llamada pide dos llamadas más (F(n-1) y F(n-2)), repitiendo muchos cálculos. Eso hace que el tiempo explote al aumentar n; en tus datos se nota mucho en $n = 25$y $n = 30$.

Búsqueda Lineal — Iterativo

Revisa elemento por elemento hasta encontrar lo que buscas o terminar la lista. Simple y directo: si la lista es más larga, tardará proporcionalmente más.

Búsqueda Lineal — Recursivo

Hace lo mismo que la iterativa pero usando llamadas recursivas para avanzar en la lista. Funciona igual en tiempo, aunque usa más memoria por la pila de llamadas.

Burbuja — Iterativo

Compara pares adyacentes y los intercambia hasta que todo queda ordenado; repite varias pasadas. Por eso, cuando duplicas $n$el tiempo crece mucho más (aprox. 4× al duplicar n), y en tus mediciones se ve claro en 5 000 y 10 000.

Burbuja — Recursivo

Implementa la misma idea de burbuja, pero con llamadas recursivas en lugar de bucles. El comportamiento en tiempo es similar al iterativo, con la sobrecarga extra de la recursión.

A continuación, se mostrarán capturas de fragmentos del código las mas interesantes para mi persona

```java
import algorithms.BusquedaLineal;
import algorithms.Factorial;
import algorithms.Fibonacci;
import algorithms.OrdenamientoBurbuja;
import benchmark.Medidor;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import utils.GeneradorDatos;
```

En esta tenemos la importación de todas las demás clases para que Main pueda funcionar correctamente

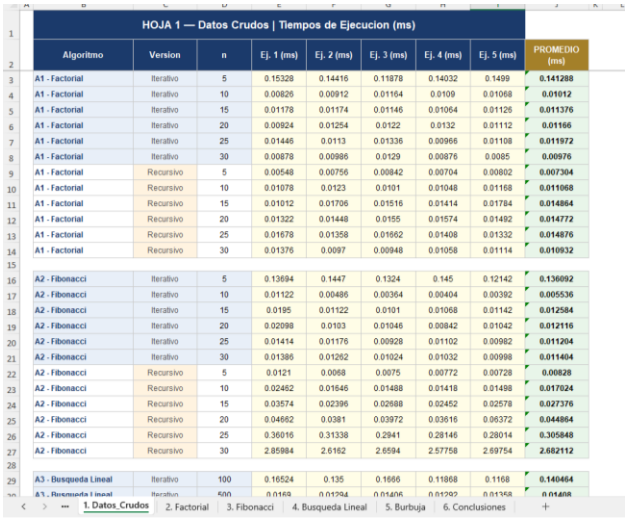
```java
public class Main {
    public static void main(String[] args) {
        new File("resultados").mkdirs();

        try (FileWriter fw = new FileWriter("resultados/tiempos.csv")) {
            fw.write("Algoritmo,Version,n,Tiempo_ms\n");

            int[] nsFactorial = {5, 10, 15, 20, 25, 30};
            for (int n : nsFactorial) {
                double tIter = Medidor.medirPromedioMillis(() -> Factorial.factorialIterativo(n));
                System.out.println("Medición: Factorial Iterativo n=" + n + " tiempo=" + tIter);
                fw.write(String.format("Factorial,Iterativo,%d,%.6f%n", n, tIter));

                double tRec = Medidor.medirPromedioMillis(() -> Factorial.factorialRecursivo(n));
                System.out.println("Medición: Factorial Recursivo n=" + n + " tiempo=" + tRec);
```

En esta captura podemos ver los nsfactorial donde están asignados los numero que solicitaron estos pueden ser modificados si así sea requerido.

```
        }

        int[] nsLinealBurbuja = {100, 500, 1000, 5000, 10000};
        for (int n : nsLinealBurbuja) {
            int[] base = GeneradorDatos.generarArregloAleatorio(n, n * 10);
            int target = base[n / 2];

            final int[] copiaBuscarIter = GeneradorDatos.clonar(base);
            double tBuscarIter = Medidor.medirPromedioMillis(() ->
                    BusquedaLineal.buscarIterativo(copiaBuscarIter, target));
            System.out.println("Medición: BusquedaLineal Iterativo n=" + n + " tiempo=" + tBuscarIter);
            fw.write(String.format(format: "BusquedaLineal,Iterativo,%d,%.6f%n", n, tBuscarIter));

            final int[] copiaBuscarRec = GeneradorDatos.clonar(base);
            double tBuscarRec = Medidor.medirPromedioMillis(() ->
                    BusquedaLineal.buscarRecursivo(copiaBuscarRec, target));
            System.out.println("Medición: BusquedaLineal Recursivo n=" + n + " tiempo=" + tBuscarRec);
            fw.write(String.format(format: "BusquedaLineal,Recursivo,%d,%.6f%n", n, tBuscarRec));

            final int[] copiaBurbujaIter = GeneradorDatos.clonar(base);
            double tBurbujaIter = Medidor.medirPromedioMillis(() -> {
                OrdenamientoBurbuja.burbujaIterativo(copiaBurbujaIter);
            });
            System.out.println("Medición: Burbuja Iterativo n=" + n + " tiempo=" + tBurbujaIter);
            fw.write(String.format(format: "Burbuja,Iterativo,%d,%.6f%n", n, tBurbujaIter));

            final int[] copiaBurbujaRec = GeneradorDatos.clonar(base);
            double tBurbujaRec = Medidor.medirPromedioMillis(() -> {
                OrdenamientoBurbuja.burbujaRecursivo(copiaBurbujaRec);
            });
            System.out.println("Medición: Burbuja Recursivo n=" + n + " tiempo=" + tBurbujaRec);
            fw.write(String.format(format: "Burbuja,Recursivo,%d,%.6f%n", n, tBurbujaRec));
        }

        System.out.println(x: "Mediciones completadas. Archivo: resultados/tiempos.csv");

    } catch (IOException e) {
        System.err.println("Error escribiendo resultados: " + e.getMessage());
        e.printStackTrace();
    }
}
```

Y en esta captura se muestra el resto del codigo que da sentido a la busqueda lineal y como tambien ala medicion de burbuja y tambien el ordenamiento para que nos de los tiempos en que se tardaron en ejecutarlos

A continuacion se mostraran las capturas del documento excel para validar mis respuestas



HOJA 1 — Datos Crudos | Tiempos de Ejecucion (ms)

Algoritmo	Version	n	Ej. 1 (ms)	Ej. 2 (ms)	Ej. 3 (ms)	Ej. 4 (ms)	Ej. 5 (ms)	PROMEDIO (ms)
A1 - Factorial	Iterativo	5	0.15328	0.14416	0.11878	0.14032	0.1499	**0.141288**
A1 - Factorial	Iterativo	10	0.00826	0.00912	0.01164	0.0109	0.01068	**0.01012**
A1 - Factorial	Iterativo	15	0.01178	0.01174	0.01146	0.01064	0.01126	**0.011376**
A1 - Factorial	Iterativo	20	0.00924	0.01254	0.0122	0.0132	0.01112	**0.01166**
A1 - Factorial	Iterativo	25	0.01446	0.0113	0.01336	0.00966	0.01108	**0.011972**
A1 - Factorial	Iterativo	30	0.00878	0.00986	0.0129	0.00876	0.0085	**0.00976**
A1 - Factorial	Recursivo	5	0.00548	0.00756	0.00842	0.00704	0.00802	**0.007304**
A1 - Factorial	Recursivo	10	0.01078	0.0123	0.0101	0.01048	0.01168	**0.011068**
A1 - Factorial	Recursivo	15	0.01012	0.01706	0.01516	0.01414	0.01784	**0.014864**
A1 - Factorial	Recursivo	20	0.01322	0.01448	0.0155	0.01574	0.01492	**0.014772**
A1 - Factorial	Recursivo	25	0.01678	0.01358	0.01662	0.01408	0.01332	**0.014876**
A1 - Factorial	Recursivo	30	0.01376	0.0097	0.00948	0.01058	0.01114	**0.010932**
A2 - Fibonacci	Iterativo	5	0.13694	0.1447	0.1324	0.145	0.12142	**0.136092**
A2 - Fibonacci	Iterativo	10	0.01122	0.00486	0.00364	0.00404	0.00392	**0.005536**
A2 - Fibonacci	Iterativo	15	0.0195	0.01122	0.0101	0.01068	0.01142	**0.012584**
A2 - Fibonacci	Iterativo	20	0.02098	0.0103	0.01046	0.00842	0.01042	**0.012116**
A2 - Fibonacci	Iterativo	25	0.01414	0.01176	0.00928	0.01102	0.00982	**0.011204**
A2 - Fibonacci	Iterativo	30	0.01386	0.01262	0.01024	0.01032	0.00998	**0.011404**
A2 - Fibonacci	Recursivo	5	0.0121	0.0068	0.0075	0.00772	0.00728	**0.00828**
A2 - Fibonacci	Recursivo	10	0.02462	0.01546	0.01488	0.01418	0.01498	**0.017024**
A2 - Fibonacci	Recursivo	15	0.03574	0.02396	0.02688	0.02452	0.02578	**0.027376**
A2 - Fibonacci	Recursivo	20	0.04662	0.0381	0.03972	0.03616	0.06372	**0.044864**
A2 - Fibonacci	Recursivo	25	0.36016	0.31338	0.2941	0.28146	0.28014	**0.305848**
A2 - Fibonacci	Recursivo	30	2.85984	2.6162	2.6594	2.57758	2.69754	**2.682112**
A3 - Busqueda Lineal	Iterativo	100	0.16524	0.135	0.1666	0.11868	0.1168	**0.140464**
A3 - Busqueda Lineal	Iterativo	500	0.0169	0.01294	0.01406	0.01292	0.01358	**0.01408**

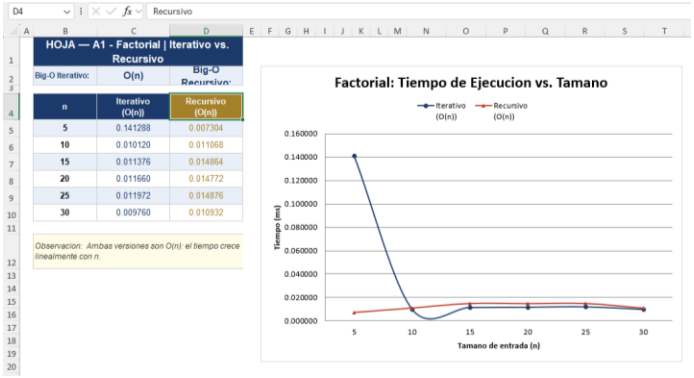
1. Datos_Crudos | 2. Factorial | 3. Fibonacci | 4. Busqueda Lineal | 5. Burbuja | 6. Conclusiones

Datos crudos de el algoritmo factorial y Fibinacci

HOJA 1 — Datos Crudos | Tiempos de Ejecucion (ms)								
Algoritmo	Version	n	Ej. 1 (ms)	Ej. 2 (ms)	Ej. 3 (ms)	Ej. 4 (ms)	Ej. 5 (ms)	PROMEDIO (ms)
A3 - Busqueda Lineal	Iterativo	100	0.16524	0.135	0.1666	0.11868	0.1168	**0.140464**
A3 - Busqueda Lineal	Iterativo	500	0.0169	0.01294	0.01406	0.01292	0.01358	**0.01408**
A3 - Busqueda Lineal	Iterativo	1000	0.01994	0.01926	0.01676	0.01316	0.01528	**0.01688**
A3 - Busqueda Lineal	Iterativo	5000	0.0393	0.04842	0.03886	0.0377	0.03636	**0.040128**
A3 - Busqueda Lineal	Iterativo	10000	0.06938	0.07376	0.0683	0.02588	0.06884	**0.061232**
A3 - Busqueda Lineal	Recursivo	100	0.0082	0.00708	0.00632	0.00632	0.00684	**0.006952**
A3 - Busqueda Lineal	Recursivo	500	0.0273	0.02304	0.02112	0.02388	0.02432	**0.023932**
A3 - Busqueda Lineal	Recursivo	1000	0.01938	0.0184	0.01314	0.01174	0.01286	**0.015104**
A3 - Busqueda Lineal	Recursivo	5000	0.03516	0.0564	0.0382	0.03508	0.04284	**0.041536**
A3 - Busqueda Lineal	Recursivo	10000	0.02814	0.02888	0.02764	0.01494	0.02506	**0.024932**
A4 - Burbuja	Iterativo	100	0.17944	0.17708	0.17132	0.00632	0.16912	**0.140656**
A4 - Burbuja	Iterativo	500	0.4504	0.42824	0.42056	0.3866	0.41322	**0.419804**
A4 - Burbuja	Iterativo	1000	0.10688	0.34188	0.1305	0.12866	0.13556	**0.168696**
A4 - Burbuja	Iterativo	5000	1.51076	1.64752	1.52182	1.63886	1.51062	**1.565916**
A4 - Burbuja	Iterativo	10000	5.6404	5.53038	5.6113	5.9487	5.5692	**5.659996**
A4 - Burbuja	Recursivo	100	0.03908	0.03774	0.03662	0.03644	0.0315	**0.036276**
A4 - Burbuja	Recursivo	500	0.11124	0.14702	0.16332	0.13562	0.17974	**0.147388**
A4 - Burbuja	Recursivo	1000	0.47768	0.53876	0.01314	0.50692	0.4991	**0.40712**
A4 - Burbuja	Recursivo	5000	1.41766	1.4754	1.32104	1.49252	1.43348	**1.42802**
A4 - Burbuja	Recursivo	10000	5.50232	5.3926	5.12078	5.50736	5.3575	**5.376112**

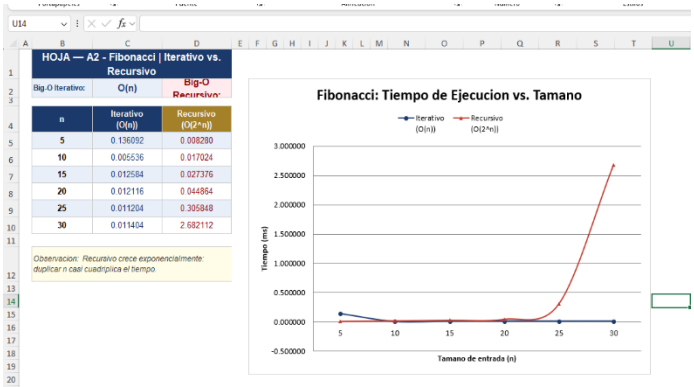
Y aquí tenemos los datos crudos de la Búsqueda lineal y Medición Burbuja

A continuación, se presentarán capturas de las tablas graficas que dieron después de agregar los datos crudos en la tabla de Excel:



HOJA — A1 - Factorial | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:
n	Iterativo (O(n))	Recursivo (O(n))
5	0.141288	0.007304
10	0.010120	0.011068
15	0.011376	0.014864
20	0.011660	0.014772
25	0.011972	0.014876
30	0.009760	0.010932

Observacion: Ambas versiones son O(n): el tiempo crece linealmente con n.



HOJA — A2 - Fibonacci | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:
n	Iterativo (O(n))	Recursivo (O(2^n))
5	0.136092	0.008280
10	0.005536	0.017024
15	0.012584	0.027376
20	0.012116	0.044864
25	0.011204	0.305848
30	0.011404	2.682112

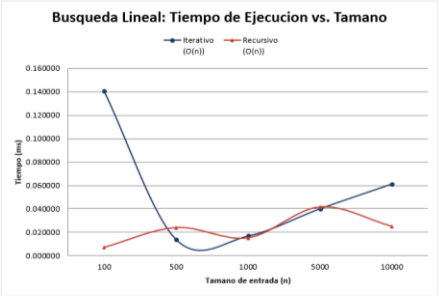
Observacion: Recursivo crece exponencialmente: duplicar n casi cuadriplica el tiempo.

HOJA — A3 - Busqueda Lineal | Iterativo vs. Recursivo

Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
100	0.140464	0.006952
500	0.014080	0.023932
1000	0.016880	0.015104
5000	0.040128	0.041536
10000	0.061232	0.024932

Obaervacion: Ambas O(n): al multiplicar n x10, el tiempo tambien x10 aprox.

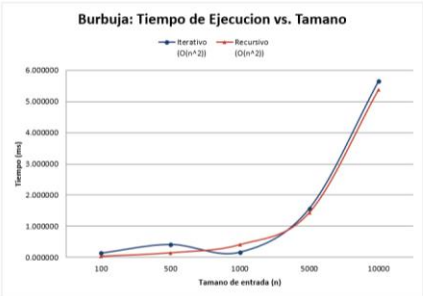


HOJA — A4 - Burbuja | Iterativo vs. Recursivo

Big-O Iterativo:	O(n^2)	Big-O Recursivo:

n	Iterativo (O(n^2))	Recursivo (O(n^2))
100	0.140656	0.036276
500	0.419804	0.147388
1000	0.168696	0.407120
5000	1.565916	1.428020
10000	5.659996	5.376112

Observacion: Ambas O(n^2): al x10 n, el tiempo aumenta aprox x100.



D7 | O(2^n)

HOJA 6 — Conclusiones | Big-O Experimental vs. Teorico

Algoritmo	Version	Big-O Experimental (de tu grafica)	Big-O Teorico	Coincide?	Justificacion breve
A1 - Factorial	Iterativo	O(n)	O(n)	SI	Tiempos crecen de forma lineal con n; diferencias pequeñas entre repeticiones.
A1 - Factorial	Recursivo	O(n)	O(n)	SI	Comportamiento lineal similar al iterativo para los n probados.
A2 - Fibonacci	Iterativo	O(n)	O(n)	SI	Tiempos estables y proporcionales a n; no explotan con n.
A2 - Fibonacci	Recursivo	O(2^n)	O(2^n)	SI	Tiempos crecen exponencialmente; n=30 tarda varios segundos.
A3 - Busqueda Lineal	Iterativo	O(n)	O(n)	SI	Tiempo aumenta aproximadamente proporcional a n; resultados coherentes.
A3 - Busqueda Lineal	Recursivo	O(n)	O(n)	SI	Comportamiento similar al iterativo; variación baja entre repeticiones.
A4 - Burbuja	Iterativo	O(n^2)	O(n^2)	SI	Tiempos aumentan rápidamente con n; 10×n = 100× tiempo en promedio.
A4 - Burbuja	Recursivo	O(n^2)	O(n^2)	SI	Comportamiento cuadrático claro; tiempos grandes para n=5000–10000.

Guia para interpretar tus graficas:	
Linea recta en grafica	→ Big-O es probablemente O(1) o O(n)
Curva suave ascendente	→ Big-O es probablemente O(n) o O(log n)
Curva que sube rapido	→ Big-O es probablemente O(n^2)
Curva que explota (vertical)	→ Big-O es probablemente O(2^n)

Análisis y Conclusiones: Iterativo vs Recursivo y Big-O

Factorial

- Iterativo vs Recursivo: Ambos métodos muestran tiempos muy pequeños y crecen de forma proporcional con n. La diferencia práctica es mínima: el iterativo usa un bucle sencillo, mientras que el recursivo depende de llamadas en pila.
- Big-O: En ambos casos el crecimiento es O(n), lo que coincide con la teoría.

Fibonacci

- Iterativo vs Recursivo: El iterativo mantiene tiempos bajos y escala de manera lineal. El recursivo, en cambio, repite muchos cálculos y los tiempos se disparan conforme aumenta n. En tus mediciones, para $n = 30$el recursivo tarda varios segundos, mientras que el iterativo sigue siendo rápido.
- Big-O: Iterativo es O(n) y coincide con la teoría. El recursivo es O(2^n), lo que se confirma experimentalmente por el crecimiento exponencial observado.

Búsqueda Lineal

- Iterativo vs Recursivo: Ambos recorren la lista elemento por elemento. El iterativo lo hace con un bucle, el recursivo con llamadas sucesivas. Los tiempos son muy similares, aunque el recursivo usa más memoria por la pila.
- Big-O: En ambos casos es O(n), y tus datos muestran que al aumentar $n$el tiempo crece proporcionalmente.

Ordenamiento Burbuja

- Iterativo vs Recursivo: Ambos comparan e intercambian elementos repetidamente. El iterativo usa bucles, el recursivo divide el problema en llamadas sucesivas. Los tiempos son grandes en ambos casos, especialmente para $n = 5000$y $n = 10000$.
- Big-O: En ambos casos es O(n²), y tus mediciones confirman que al duplicar $n$el tiempo se multiplica aproximadamente por 4, lo que coincide con la teoría.

Conclusión

Los resultados experimentales coinciden con las expectativas teóricas de Big-O:

- Factorial y Búsqueda Lineal muestran crecimiento lineal (O(n)).
- Burbuja evidencia crecimiento cuadrático ($O(n^2)$).
- Fibonacci recursivo confirma el crecimiento exponencial (O(2^n)), siendo el más lento y poco práctico para valores grandes.

Esto demuestra que la elección entre versión iterativa y recursiva no siempre afecta la complejidad teórica, pero sí puede impactar en la eficiencia práctica, especialmente en algoritmos como Fibonacci donde la recursión sin optimización resulta muy costosa.

Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th ed.). Wiley.